SPC CAT 1

C Programming Question Bank with Solutions

First Trimester | Course 26CSE0105

A broad practice set with worked solutions, built from the shared SPC lecture material. It covers problem solving, algorithms, flowcharts, C fundamentals, input/output, data types, operators, macros, libraries, pointers, debugging, and testing.

How to use this bank: try each prompt first, then compare your work with the suggested solution immediately below it. For programming questions, identify input, process, and output; write an algorithm when useful; then code, trace, and test. Questions are marked Basic, Standard, or Challenge.

Scope note: no finite list can contain every possible wording or number variation. This bank covers every topic visible in the shared nine-lecture SPC material and includes common exam-style variants. The decision and loop section is explicitly tied to exercises in the notes; use C if/else and loop syntax only if your CAT 1 has covered it.

Question map

1. Problem-solving, algorithms and flowcharts - Questions 1-10 (10 questions)

2. C program structure and console input/output - Questions 11-22 (12 questions)

3. Variables, data types and type conversion - Questions 23-33 (11 questions)

4. Operators, expressions and evaluation - Questions 34-49 (16 questions)

5. Decision and repetition exercises from the notes - Questions 50-62 (13 questions)

6. Headers, macros, functions and pointers - Questions 63-75 (13 questions)

7. Debugging, tracing and testing - Questions 76-89 (14 questions)

8. Integrated CAT-style coding challenges - Questions 90-99 (10 questions)

9. Two short mock CAT 1 practice sets - Questions 100-109 (10 questions)

Total: 109 questions, including two mock CAT 1 sets.

1. Problem-solving, algorithms and flowcharts

Turn the problem statement into a clear plan before writing C. Questions marked 'design' can be answered as an algorithm, pseudocode, flowchart, or all three as instructed.

Q1. Design | Sum of three integers [Basic]

Read three integers, calculate their sum, and display it. Write the input, process, output, and a numbered algorithm.

Suggested solution

START; read a, b, c; sum = a + b + c; display sum; STOP.

Q2. Design | Largest of three [Basic]

Read three integers and determine the largest. Write pseudocode that also handles ties correctly.

Suggested solution

Read a, b, c. Set largest = a. If b > largest, set largest = b. If c > largest, set largest = c. Display largest. Using > rather than a chain that assumes distinct values handles ties.

Q3. Design | Rectangle report [Basic]

Given a rectangle's length and width, produce its perimeter and area. State assumptions, inputs, outputs, and the algorithm.

Suggested solution

INPUT length, width. area = length * width. perimeter = 2 * (length + width). OUTPUT area, perimeter. Assume dimensions are non-negative.

Q4. Flowchart | Voting eligibility [Basic]

Draw a flowchart that reads a person's age and displays whether the person is eligible to vote at age 18 or above.

Suggested solution

Flow: Start -> read age -> decision 'age >= 18?' -> Yes: display Eligible; No: display Not eligible -> Stop.

Q5. Flowchart | Even or odd [Basic]

Draw a flowchart to read an integer and classify it as even or odd. Show the decision condition.

Suggested solution

Flow: Start -> read n -> decision 'n % 2 == 0?' -> Yes: display Even; No: display Odd -> Stop.

Q6. Pseudocode | Sign of a number [Basic]

Write pseudocode to classify an entered number as positive, negative, or zero.

Suggested solution

BEGIN; INPUT n; IF n > 0 OUTPUT Positive; ELSE IF n < 0 OUTPUT Negative; ELSE OUTPUT Zero; END.

Q7. Design | Five subject marks [Basic]

Read marks in five subjects and calculate the total and average. List the input, process, output, and test values you would use.

Suggested solution

total = m1 + m2 + m3 + m4 + m5; average = total / 5.0. Inputs are the five marks; outputs are total and average. Test all zeroes and a mix of low and high marks.

Q8. Flowchart | Factorial [Standard]

Draw a repetition flowchart to calculate n! for a non-negative integer n. Make sure the flow handles n = 0.

Suggested solution

BEGIN; INPUT n; fact = 1; FOR i = 1 TO n: fact = fact * i; OUTPUT fact; END. For n = 0 the loop runs zero times and fact remains 1.

Q9. Trace | Sum from 1 to 5 [Standard]

Trace a counter-based algorithm that starts sum at 0 and adds the values 1 through 5. Show the counter and sum after each repetition.

Suggested solution

The accumulator values after adding 1, 2, 3, 4, 5 are 1, 3, 6, 10, 15. The final sum is 15; the counter advances from 1 through 6 before the loop ends.

Q10. Translate | Largest of three [Standard]

Convert the largest-of-three algorithm from Question 2 into a complete C program. Test it with three equal values and with a negative largest value.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int a, b, c, largest;
    scanf("%d%d%d", &a, &b, &c);
    largest = a;
    if (b > largest) largest = b;
    if (c > largest) largest = c;
    printf("Largest = %d\n", largest);
    return 0;
}
```

2. C program structure and console input/output

Write complete C programs unless the prompt asks for a fragment. Use standard headers, int main(void), clear variable names, matching format specifiers, and return 0.

Q11. First program | Hello, C [Basic]

Write a complete C program that prints 'Hello, C!' on one line and your course name on the next line.

Suggested solution

```
#include <stdio.h>
int main(void) {
    printf("Hello, C!\nFirst Trimester\n");
    return 0;
}
```

Q12. Formatted output | Student card [Basic]

Read a student's name and age, then print a two-line student card with labels aligned using tabs and newlines. Use a safe line-input function for the name.

Suggested solution

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char name[80];
    int age;
    printf("Age: ");
    scanf("%d", &age);
    getchar();
    printf("Name: ");
    if (fgets(name, sizeof name, stdin) == NULL) return 1;
    name[strcspn(name, "\n")] = '\0';
    printf("Student\t%s\nAge\t%d\n", name, age);
    return 0;
}
```

Q13. Input/output | Two-number arithmetic [Basic]

Read two integers and print their sum, difference, product, and quotient. State how your program handles a zero second input.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int a, b;
    if (scanf("%d%d", &a, &b) != 2) return 1;
    printf("Sum = %d\nDifference = %d\nProduct = %d\n", a+b, a-b, a*b);
    if (b != 0) printf("Quotient = %d\n", a/b);
    else puts("Quotient undefined: divisor is zero.");
    return 0;
}
```

Q14. Input/output | Three-number summary [Basic]

Read three numbers and display their sum and arithmetic mean to two decimal places.

Suggested solution

```
#include <stdio.h>
int main(void) {
    double a, b, c;
    scanf("%lf%lf%lf", &a, &b, &c);
    printf("Sum = %.2f\nMean = %.2f\n", a+b+c, (a+b+c)/3.0);
    return 0;
}
```

Q15. Input/output | Swap with a temporary [Basic]

Read two integers, exchange their values using a third variable, and print the values before and after the exchange.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int a, b, temp;
    scanf("%d%d", &a, &b);
    printf("Before: a=%d b=%d\n", a, b);
    temp = a; a = b; b = temp;
    printf("After:  a=%d b=%d\n", a, b);
    return 0;
}
```

Q16. Input/output | Circle measurements [Standard]

Read a circle's radius and display its diameter, circumference, and area. Use floating-point calculations and a named PI constant.

Suggested solution

```
#include <stdio.h>
#define PI 3.141592653589793
int main(void) {
    double r;
    scanf("%lf", &r);
    printf("Diameter = %.2f\nCircumference = %.2f\nArea = %.2f\n",
           2*r, 2*PI*r, PI*r*r);
    return 0;
}
```

Q17. Input/output | Travel calculation [Basic]

Read distance in kilometres and time in hours. Calculate and display average speed in kilometres per hour. Reject or explain zero time.

Suggested solution

```
#include <stdio.h>
int main(void) {
    double km, hours;
    scanf("%lf%lf", &km, &hours);
    if (hours <= 0) puts("Time must be positive.");
    else printf("Average speed = %.2f km/h\n", km/hours);
    return 0;
}
```

Q18. Input/output | Minutes to hours [Standard]

Read a non-negative number of minutes and display the equivalent number of whole hours and remaining minutes.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int minutes;
    scanf("%d", &minutes);
    if (minutes < 0) return 1;
    printf("%d hours %d minutes\n", minutes/60, minutes%60);
    return 0;
}
```

Q19. Input/output | Mixed data [Basic]

Read an integer, a decimal value, and a single character, then print each on a separate labeled line using the correct format specifier.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int count; float value; char category;
    if (scanf("%d %f %c", &count, &value, &category) != 3) return 1;
    printf("Count = %d\nValue = %.2f\nCategory = %c\n", count, value, category);
    return 0;
}
```

Q20. Character I/O | Echo one character [Basic]

Read one character using getchar(), then display it using putchar(). Add a short prompt before reading.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int ch;
    printf("Enter a character: ");
    ch = getchar();
    if (ch != EOF) { printf("You entered: "); putchar(ch); }
    return 0;
}
```

Q21. String I/O | Greeting [Basic]

Read a full name that may contain spaces using fgets(), then display 'Hello' followed by the name. Avoid the unsafe gets() function.

Suggested solution

```
#include <stdio.h>
int main(void) {
    char name[100];
    printf("Enter your full name: ");
    if (fgets(name, sizeof name, stdin) == NULL) return 1;
    printf("Hello %s", name);
    return 0;
}
```

Q22. Escape sequences | Quoted message [Basic]

Write a program that prints a three-line message. Include a tab between a label and value, quotation marks around one word, and a backslash in a sample file path.

Suggested solution

```
#include <stdio.h>
int main(void) {
    printf("Label:\tValue\n");
    printf("He said, \"Ready\"\n");
    printf("Path: C:\\SPC\\CAT1\n");
    return 0;
}
```

3. Variables, data types and type conversion

Choose a suitable type for every value. Where compiler-dependent sizes matter, use sizeof() in the program instead of assuming one fixed answer.

Q23. Types | sizeof report [Basic]

Write a program that displays sizeof(char), sizeof(int), sizeof(float), and sizeof(double), with clear labels. Do not hard-code the sizes.

Suggested solution

```
#include <stdio.h>
int main(void) {
    printf("char: %zu byte(s)\n", sizeof(char));
    printf("int: %zu byte(s)\n", sizeof(int));
    printf("float: %zu byte(s)\n", sizeof(float));
    printf("double: %zu byte(s)\n", sizeof(double));
    return 0;
}
```

Q24. Conversion | Accurate average [Basic]

Read three integer marks and print their average with two digits after the decimal point. Use an explicit conversion or a floating-point divisor to avoid integer truncation.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int a, b, c;
    scanf("%d%d%d", &a, &b, &c);
    printf("Average = %.2f\n", (a+b+c)/3.0);
    return 0;
}
```

Q25. Trace | Integer and real division [Standard]

Write a short program that stores 7 / 2 in an int and 7.0 / 2 in a double, then prints both. Explain the different results in a comment.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int whole = 7 / 2;
    double real = 7.0 / 2;
    printf("%d %.1f\n", whole, real); /* 3 and 3.5 */
    return 0;
}
```

Q26. Conversion | Repair truncation [Basic]

The expression (a + b) / 2 prints 4 when a = 3 and b = 6, but the required answer is 4.5. Rewrite the expression so the average is calculated as a floating-point value.

Suggested solution

Use (a + b) / 2.0 or ((double)a + b) / 2.0. For 3 and 6, the result is 4.50 rather than the truncated integer 4.

Q27. Types | Read and report [Basic]

Read a whole-number count, a decimal unit price, and a category character. Store each in an appropriate type and print them with matching format specifiers.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int count; float price; char category;
    if (scanf("%d %f %c", &count, &price, &category) != 3) return 1;
    printf("Count=%d Price=%.2f Category=%c\n", count, price, category);
    return 0;
}
```

Q28. Types | Signed and unsigned counts [Standard]

Write a short C program that declares a signed integer and an unsigned integer, assigns them valid sample values, and prints them. Add a comment describing when an unsigned type is appropriate.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int signed_count = -3;
    unsigned int available = 12u;
    printf("signed=%d unsigned=%u\n", signed_count, available);
    return 0;
}
```

Q29. Conversion | Seconds breakdown [Standard]

Read a non-negative number of seconds and convert it to whole hours, remaining minutes, and remaining seconds using integer division and the remainder operator.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int total, h, m, s;
    scanf("%d", &total);
    if (total < 0) return 1;
    h = total / 3600;
    m = (total % 3600) / 60;
    s = total % 60;
    printf("%d h %d m %d s\n", h, m, s);
    return 0;
}
```

Q30. Conversion | Centimetres and metres [Basic]

Read a length in centimetres and print the equivalent metres as a decimal value. Then read metres and display centimetres.

Suggested solution

```
#include <stdio.h>
int main(void) {
    double cm, metres;
    scanf("%lf", &cm);
    printf("%.2f cm = %.2f m\n", cm, cm/100.0);
    scanf("%lf", &metres);
    printf("%.2f m = %.2f cm\n", metres, metres*100.0);
    return 0;
}
```

Q31. Calculation | Simple interest [Basic]

Read principal, annual rate, and time in years. Calculate simple interest using I = P * R * T / 100 and display the result to two decimal places.

Suggested solution

```
#include <stdio.h>
int main(void) {
    double p, r, t;
    scanf("%lf%lf%lf", &p, &r, &t);
    printf("Simple interest = %.2f\n", p*r*t/100.0);
    return 0;
}
```

Q32. Calculation | Body mass index [Standard]

Read weight in kilograms and height in metres. Calculate BMI as weight divided by height squared, and print it to one decimal place. State how you avoid division by zero.

Suggested solution

```
#include <stdio.h>
int main(void) {
    double weight, height;
    scanf("%lf%lf", &weight, &height);
    if (height <= 0) puts("Height must be positive.");
    else printf("BMI = %.1f\n", weight/(height*height));
    return 0;
}
```

Q33. Calculation | Salary components [Standard]

Read basic salary, calculate a 20% housing allowance and a 10% travel allowance, then display each allowance and gross salary. Use floating-point values.

Suggested solution

```
#include <stdio.h>
int main(void) {
    double basic, hra, travel;
    scanf("%lf", &basic);
    hra = basic * 0.20;
    travel = basic * 0.10;
    printf("HRA = %.2f\nTravel = %.2f\nGross = %.2f\n",
           hra, travel, basic+hra+travel);
    return 0;
}
```

4. Operators, expressions and evaluation

For trace questions, show intermediate values. C scans tokens from left to right, but operator precedence and associativity determine how an expression is grouped.

Q34. Trace | Precedence [Basic]

Predict the output and show the evaluation order: int a = 5, b = 3, c = 2; printf("%d\n", a + b * c);

Suggested solution

Multiplication is done before addition: 3 * 2 = 6, then 5 + 6 = 11. Output: 11.

Q35. Trace | Integer division and remainder [Basic]

For int a = 17, b = 5;, predict the values printed by a / b and a % b. Explain why the quotient has no fractional part.

Suggested solution

Integer division: 17 / 5 = 3; remainder: 17 % 5 = 2.

Q36. Trace | Relational results [Basic]

For a = 25 and b = 5, write a program that prints the results of a < b, a > b, a <= b, a >= b, a == b, and a != b. Predict each printed 0 or 1.

Suggested solution

The six results in order are 0, 1, 0, 1, 0, 1. Relational expressions print 1 for true and 0 for false.

Q37. Trace | Logical operators [Standard]

For int a = 25, b = 5;, print and predict a && b, a || b, and !a. Remember that any non-zero integer is treated as true.

Suggested solution

For non-zero a and b: a && b is 1, a || b is 1, and !a is 0.

Q38. Operators | Compound assignment [Standard]

Start with a = 25 and b = 5. Apply a += b, a -= b, a *= b, a /= b, and a %= b in that order, printing a after every operation. Record the values.

Suggested solution

In order, a becomes 30, 25, 125, 25, and 0.

Q39. Trace | Pre-increment and post-increment [Standard]

Write a short program that demonstrates the difference between ++x and x++ in separate statements. Predict each output and show the final value of x.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int x = 1;
    printf("%d\n", ++x); /* prints 2; x is now 2 */
    x = 1;
    printf("%d\n", x++); /* prints 1; x becomes 2 afterward */
    printf("final x = %d\n", x);
    return 0;
}
```

Q40. Ternary | Larger of two [Basic]

Read two integers and use the conditional (ternary) operator to select and print the larger value. Include an output for equal inputs.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int a, b, largest;
    scanf("%d%d", &a, &b);
    largest = (a > b) ? a : b;
    printf("Largest = %d\n", largest);
    return 0;
}
```

Q41. Ternary | Positive or non-positive [Basic]

Read an integer and use a conditional operator to display whether it is positive or not positive.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);
    printf("%s\n", (n > 0) ? "Positive" : "Not positive");
    return 0;
}
```

Q42. Bitwise | AND and OR [Standard]

For a = 12 and b = 5, print a & b and a | b. Show the binary representation of each operand and result in comments.

Suggested solution

12 = 1100 and 5 = 0101 in binary. 1100 & 0101 = 0100 = 4; 1100 | 0101 = 1101 = 13.

Q43. Bitwise | Shifts [Standard]

For a = 12, print a << 2 and a >> 2. Explain the binary shift that produces each result for this positive value.

Suggested solution

12 is 1100 in binary. Left shift by 2 gives 110000 = 48; right shift by 2 gives 0011 = 3.

Q44. Operators | Eligibility condition [Standard]

Read age and citizenship status as a character ('Y' or 'N'). Print the result of a logical expression that is true only when age is at least 18 and citizenship is 'Y'.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int age; char citizen;
    scanf("%d %c", &age, &citizen);
    if (age >= 18 && citizen == 'Y') puts("Eligible");
    else puts("Not eligible");
    return 0;
}
```

Q45. Operators | Remainder check [Basic]

Read an integer and print its remainder after division by 2, 3, and 5. Then state which of those divisors divide it exactly.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int n;
    scanf("%d", &n);
    printf("Remainders: %d %d %d\n", n%2, n%3, n%5);
    printf("Divisible by 2/3/5: %d %d %d\n",
           n%2==0, n%3==0, n%5==0);
    return 0;
}
```

Q46. Challenge | Swap without a third variable [Challenge]

Exchange two small, non-overflowing integers using arithmetic assignment operators only. Print the values before and after. Explain why a temporary variable is usually safer.

Suggested solution

```
/* For small values only: a+b must not overflow. */
int a = 7, b = 4;
a = a + b;
b = a - b;
a = a - b;
/* Now a is 4 and b is 7. A temporary variable is safer. */
```

Q47. Trace | Mixed types [Standard]

Let int count = 5; and float price = 2.5f;. Trace count * price, (int)price, and (float)count / 2, printing each with a suitable format specifier.

Suggested solution

count * price is 12.5; (int)price is 2; (float)count / 2 is 2.5. Print them with %g or suitable %f formats.

Q48. Bitwise | Test one bit [Challenge]

Read a non-negative integer n and a bit position k. Use a bit mask and the bitwise AND operator to determine whether bit k is set; print the result.

Suggested solution

```
#include <stdio.h>
int main(void) {
    unsigned int n; unsigned k;
    scanf("%u%u", &n, &k);
    if (k >= sizeof n * 8u) return 1;
    puts((n & (1u << k)) ? "Set" : "Not set");
    return 0;
}
```

Q49. Bitwise | Set, clear and toggle [Challenge]

Read an unsigned integer and a bit position. Display the value after setting, clearing, and toggling that bit using bitwise operators.

Suggested solution

```
#include <stdio.h>
int main(void) {
    unsigned int n, k, mask;
    scanf("%u%u", &n, &k);
    if (k >= sizeof n * 8u) return 1;
    mask = 1u << k;
    printf("set=%u clear=%u toggle=%u\n", n|mask, n&~mask, n^mask);
    return 0;
}
```

5. Decision and repetition exercises from the notes

These prompts are based on the explicit flowchart and pseudocode exercises in the supplied SPC material. Write the C version if if/else and loops are in your CAT 1 coverage; otherwise submit the requested algorithm, pseudocode, or flowchart.

Q50. Decision | Even or odd [Basic]

Read an integer and use the remainder operator with a decision to display whether it is even or odd. Test zero and a negative integer too.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int n; scanf("%d", &n);
    puts(n % 2 == 0 ? "Even" : "Odd");
    return 0;
}
```

Q51. Decision | Positive, negative, or zero [Basic]

Read an integer and display exactly one of: Positive, Negative, or Zero.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int n; scanf("%d", &n);
    if (n > 0) puts("Positive");
    else if (n < 0) puts("Negative");
    else puts("Zero");
    return 0;
}
```

Q52. Decision | Largest of two [Basic]

Read two different integers and display the larger. Then adapt the program to handle equal inputs without giving a misleading answer.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int a, b; scanf("%d%d", &a, &b);
    if (a > b) printf("%d is larger\n", a);
    else if (b > a) printf("%d is larger\n", b);
    else puts("The values are equal.");
    return 0;
}
```

Q53. Decision | Largest of three [Basic]

Read three integers and display the largest. Your logic must work when two or all three values are equal.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int a, b, c, largest;
    scanf("%d%d%d", &a, &b, &c);
    largest = a;
    if (b > largest) largest = b;
    if (c > largest) largest = c;
    printf("Largest = %d\n", largest);
    return 0;
}
```

Q54. Decision | Vowel or consonant [Standard]

Read one alphabetic character and classify it as a vowel or consonant. Make the comparison work for both uppercase and lowercase vowels.

Suggested solution

```
#include <stdio.h>
int main(void) {
    char ch; scanf(" %c", &ch);
    if (ch>='A' && ch<='Z') ch = (char)(ch-'A'+'a');
    if (ch<'a' || ch>'z') puts("Not an alphabetic letter");
    else if (ch=='a'||ch=='e'||ch=='i'||ch=='o'||ch=='u') puts("Vowel");
    else puts("Consonant");
    return 0;
}
```

Q55. Decision | Voting eligibility [Basic]

Read age and display whether the person is eligible to vote at age 18 or above. Include a test for age 17 and age 18.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int age; scanf("%d", &age);
    puts(age >= 18 ? "Eligible" : "Not eligible");
    return 0;
}
```

Q56. Repetition | Factorial [Standard]

Read a non-negative integer n and calculate n! using a loop. Define the result for n = 0 and avoid running the loop for a negative input.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int n; unsigned long long fact = 1;
    scanf("%d", &n);
    if (n < 0 || n > 20) { puts("Enter n from 0 to 20."); return 1; }
    for (int i=1; i<=n; ++i) fact *= (unsigned long long)i;
    printf("%llu\n", fact);
    return 0;
}
```

Q57. Repetition | Sum five inputs [Standard]

Read exactly five numbers using a repetition structure, accumulate their sum, and display the result. Do not write five separate input statements.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int value; long long sum = 0;
    for (int i=0; i<5; ++i) {
        if (scanf("%d", &value) != 1) return 1;
        sum += value;
    }
    printf("Sum = %lld\n", sum);
    return 0;
}
```

Q58. Repetition | Sum from 1 to N [Standard]

Read a positive integer N and calculate 1 + 2 + ... + N using a counter and accumulator.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int n; long long sum = 0;
    scanf("%d", &n);
    if (n < 1) return 1;
    for (int i=1; i<=n; ++i) sum += i;
    printf("Sum = %lld\n", sum);
    return 0;
}
```

Q59. Repetition | Multiplication table [Standard]

Read an integer and display its multiplication table from 1 through 10, with one equation per line.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int n; scanf("%d", &n);
    for (int i=1; i<=10; ++i) printf("%d x %d = %d\n", n, i, n*i);
    return 0;
}
```

Q60. Decision | Pass or fail [Standard]

Read a mark from 0 to 100 and display Pass if the mark is at least 50, otherwise Fail. Reject values outside the allowed range.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int mark; scanf("%d", &mark);
    if (mark < 0 || mark > 100) puts("Invalid mark");
    else if (mark >= 50) puts("Pass");
    else puts("Fail");
    return 0;
}
```

Q61. Decision | Three-way calculator [Challenge]

Read two numbers and an operator character (+, -, *, or /). Use selection logic to perform the requested operation; handle an unsupported operator and division by zero.

Suggested solution

```
#include <stdio.h>
int main(void) {
    double a, b; char op;
    if (scanf("%lf %c %lf", &a, &op, &b) != 3) return 1;
    switch (op) {
        case '+': printf("%.2f\n", a+b); break;
        case '-': printf("%.2f\n", a-b); break;
        case '*': printf("%.2f\n", a*b); break;
        case '/': if (b == 0) puts("Cannot divide by zero");
                  else printf("%.2f\n", a/b); break;
        default: puts("Unsupported operator");
    }
    return 0;
}
```

Q62. Decision | Divisible by both [Standard]

Read an integer and determine whether it is divisible by both 5 and 11. Use a combined logical condition and test a number divisible by only one of them.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int n; scanf("%d", &n);
    puts(n%5==0 && n%11==0 ? "Divisible by both" : "Not divisible by both");
    return 0;
}
```

6. Headers, macros, functions and pointers

Use the library or language feature named in each prompt. The source notes introduce these concepts; function-design problems are marked standard or challenge because exact function syntax may depend on what your instructor has covered.

Q63. Macro | GST calculation [Basic]

Define GST as a symbolic constant using #define. Read a product price and display the GST amount and final price. Use the 18% rate shown in the SPC practice example.

Suggested solution

```
#include <stdio.h>
#define GST_RATE 0.18
int main(void) {
    double price; scanf("%lf", &price);
    printf("GST Amount: %.2f\nFinal Price: %.2f\n", price*GST_RATE, price*(1+GST_RATE));
    return 0;
}
```

Q64. Macro | PI constant [Basic]

Define PI with #define, read a radius, and display a circle's area and circumference. Keep the constant name uppercase.

Suggested solution

```
#include <stdio.h>
#define PI 3.141592653589793
int main(void) {
    double r; scanf("%lf", &r);
    printf("Area=%.2f Circumference=%.2f\n", PI*r*r, 2*PI*r);
    return 0;
}
```

Q65. Library | Math calculations [Standard]

Read a number and an exponent. Use functions from math.h to display its square root, power, and absolute value. State the header required.

Suggested solution

```
#include <stdio.h>
#include <math.h>
int main(void) {
    double x, p; scanf("%lf%lf", &x, &p);
    if (x < 0) puts("Square root requires a non-negative input.");
    else printf("sqrt=%.2f\npower=%.2f\nabsolute=%.2f\n", sqrt(x), pow(x,p), fabs(x));
    return 0;
}
```

Q66. Macro | Safe square macro [Standard]

Define a macro SQUARE(x) that correctly squares an expression such as n + 1. Demonstrate it with n = 3 and use parentheses to prevent precedence errors.

Suggested solution

```
#include <stdio.h>
#define SQUARE(x) ((x) * (x))
int main(void) {
    int n=3;
    printf("%d\n", SQUARE(n+1)); /* 16 */
    return 0;
}
```

Q67. Function | Add two numbers [Standard]

Write a function that receives two integers and returns their sum. Call it from main() and display the result.

Suggested solution

```
#include <stdio.h>
int add(int a, int b) { return a+b; }
int main(void) {
    int a,b; scanf("%d%d", &a,&b);
    printf("Sum = %d\n", add(a,b));
    return 0;
}
```

Q68. Function | Largest of three [Standard]

Write a function that receives three integers and returns the largest. Call it from main() and test with tied values.

Suggested solution

```
#include <stdio.h>
int largest(int a,int b,int c) {
    int m=a; if (b>m) m=b; if (c>m) m=c; return m;
}
int main(void) {
    int a,b,c; scanf("%d%d%d", &a,&b,&c);
    printf("Largest = %d\n", largest(a,b,c));
    return 0;
}
```

Q69. Function | Rectangle calculations [Standard]

Create separate functions for the area and perimeter of a rectangle. Read the dimensions in main(), call both functions, and print their results.

Suggested solution

```
#include <stdio.h>
double area(double l,double w) { return l*w; }
double perimeter(double l,double w) { return 2*(l+w); }
int main(void) {
    double l,w; scanf("%lf%lf", &l,&w);
    printf("Area=%.2f Perimeter=%.2f\n", area(l,w), perimeter(l,w));
    return 0;
}
```

Q70. Function | Seconds conversion [Challenge]

Write a function that receives total seconds and computes hours, minutes, and seconds through output parameters. Print the results in main().

Suggested solution

```
#include <stdio.h>
void split_time(int total, int *h, int *m, int *s) {
    *h=total/3600; *m=(total%3600)/60; *s=total%60;
}
int main(void) {
    int total,h,m,s; scanf("%d", &total);
    if (total<0) return 1;
    split_time(total,&h,&m,&s);
    printf("%d:%02d:%02d\n", h,m,s);
    return 0;
}
```

Q71. Program organization | Header choices [Standard]

Write a small program that reads a value, calls sqrt(), and prints the result. Include every required standard header and add a comment explaining each one's role.

Suggested solution

```
#include <stdio.h>
#include <math.h>
int main(void) {
    double x; scanf("%lf", &x);
    if (x < 0) return 1;
    printf("sqrt = %.2f\n", sqrt(x));
    return 0;
}
```

Q72. Pointer | Address and value [Standard]

Declare an integer x and an int pointer p. Assign p the address of x, then print x, &x, p, and *p with suitable format specifiers.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int x=10; int *p=&x;
    printf("x=%d\n&x=%p\np=%p\n*p=%d\n", x,(void *)&x,(void *)p,*p);
    return 0;
}
```

Q73. Pointer | Change through a pointer [Standard]

Set an integer to 10, point p to it, then change the integer to 25 by assigning through *p. Print the variable before and after.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int x=10; int *p=&x;
    printf("Before: %d\n", x);
    *p=25;
    printf("After: %d\n", x);
    return 0;
}
```

Q74. Preprocessor | Conditional compilation [Challenge]

Write a program with a DEBUG macro and an #ifdef block that prints an extra diagnostic message only when DEBUG is defined. Keep the normal result visible in either build.

Suggested solution

```
#include <stdio.h>
/* Compile with -DDEBUG to enable the extra line. */
int main(void) {
    int x=7;
#ifdef DEBUG
    printf("Debug: x=%d\n", x);
#endif
    printf("Result = %d\n", x*x);
    return 0;
}
```

Q75. Macro or function | Compare [Challenge]

Write a macro and a function that each calculate the square of an integer. Call both with the same value and describe one safety or type-checking difference in a code comment.

Suggested solution

```
#include <stdio.h>
#define SQUARE_MACRO(x) ((x)*(x))
int square_function(int x) { return x*x; }
int main(void) {
    int n=4;
    printf("macro=%d function=%d\n", SQUARE_MACRO(n), square_function(n));
    return 0;
}
```

Both print 16 here. The function has a typed parameter and evaluates its argument once; macros are text substitution and need parentheses to avoid precedence problems.

7. Debugging, tracing and testing

For repair questions, rewrite the corrected code or describe each exact change. For testing questions, include normal, boundary, and invalid inputs where appropriate.

Q76. Fix | Missing statement terminator [Basic]

Find and correct the compilation error: #include <stdio.h> int main(void) { printf("Hello") return 0; }.

Suggested solution

Corrected line: printf("Hello");. Every C statement here must end with a semicolon.

Q77. Fix | scanf address [Basic]

This program does not reliably store input: int age; scanf("%d", age);. Rewrite the input statement correctly and explain the role of &.

Suggested solution

Correct line: scanf("%d", &age);. scanf needs the address of age so it can store the input there.

Q78. Fix | Format specifier [Basic]

Repair the program that stores a float value in price but prints it with %d. Use the correct format specifier and declaration.

Suggested solution

Declare float price; and print it with printf("%f", price); (or %.2f for two decimals). %d is for int.

Q79. Fix | Integer average [Basic]

A program reads three int marks and computes int average = (a + b + c) / 3;, but the required output includes decimals. Correct the calculation and output formatting.

Suggested solution

Use double average = (a + b + c) / 3.0; and printf("Average = %.2f", average);. The floating divisor prevents integer truncation.

Q80. Fix | Division by zero [Basic]

A program reads x and y, then prints x / y. Add input validation so it never performs integer division when y is zero.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int x,y;
    if (scanf("%d%d", &x,&y)!=2) return 1;
    if (y==0) puts("Error: divisor is zero");
    else printf("Quotient = %d\n", x/y);
    return 0;
}
```

Q81. Fix | Macro precedence [Standard]

Given #define SQR(x) x*x, the expression SQR(a + b) gives an unexpected result. Rewrite the macro safely and show the result for a = 2, b = 3.

Suggested solution

Use #define SQR(x) ((x) * (x)). For a=2 and b=3, SQR(a+b) expands safely to ((a+b)*(a+b)) and gives 25.

Q82. Classify | Three error types [Basic]

For each case, classify the error as compilation, runtime, or logical: a missing semicolon; dividing by zero; and using + where the required operation is multiplication.

Suggested solution

Missing semicolon: compilation error. Division by zero during execution: runtime error. Using + instead of the required *: logical error if the program still compiles and runs.

Q83. Trace | Arithmetic program [Basic]

Trace int a = 5, b = 10; int c = a + b * 2; and write the exact value printed by printf("%d\n", c). Show the operation order.

Suggested solution

Compute multiplication first: 10 * 2 = 20; then 5 + 20 = 25. The program prints 25.

Q84. Trace | Assignment sequence [Standard]

Starting with a = 25 and b = 5, trace a = b; a += b; a -= b; a *= b; and a /= b; in order. Record a after each statement.

Suggested solution

a becomes 5 after a=b; then 10 after +=b; then 5 after -=b; then 25 after *=b; then 5 after /=b.

Q85. Test design | Largest of three [Standard]

Create a compact test table for a largest-of-three program. Include a unique largest first value, unique largest middle value, unique largest last value, a tie, and all-negative inputs.

Suggested solution

Example table (a,b,c -> largest): (9,4,2 -> 9), (1,8,3 -> 8), (1,2,7 -> 7), (5,5,2 -> 5), (-7,-2,-9 -> -2), (4,4,4 -> 4).

Q86. Test design | Factorial [Standard]

List at least five test cases for a factorial program, including n = 0, n = 1, a typical positive value, a negative value, and a value near the chosen safe range.

Suggested solution

Suggested tests: 0! = 1; 1! = 1; 5! = 120; -1 should be rejected; 20! is within unsigned long long, while 21! is outside that type's range.

Q87. Test design | Two-number arithmetic [Standard]

Create test cases for addition, subtraction, multiplication, and division. Include zero, negative values, and a zero divisor for the division case.

Suggested solution

Tests: (8,2) for normal arithmetic; (-8,2) for negative input; (0,3) for zero; (5,0) to confirm division is rejected; and (7,2) to check integer quotient and remainder.

Q88. Review | Valid identifiers [Basic]

For each proposed name - total, 2marks, student_name, float, room-number, and _count - state whether it is a valid identifier and give the rule that decides it.

Suggested solution

Valid: total, student_name, _count. Invalid: 2marks (starts with a digit), float (keyword), room-number (hyphen is not allowed).

Q89. Improve | Readability and documentation [Basic]

Take a short program that reads length and width and calculates area. Rewrite it with descriptive variable names, indentation, and comments for the purpose and formula.

Suggested solution

```
#include <stdio.h>
int main(void) {
    double length, width, area;
    /* Calculate and display the area of a rectangle. */
    printf("Enter length and width: ");
    if (scanf("%lf%lf", &length,&width)!=2) return 1;
    area = length * width;
    printf("Area = %.2f\n", area);
    return 0;
}
```

8. Integrated CAT-style coding challenges

These longer questions combine several topics from the lecture files. State assumptions, choose suitable types, and test edge cases before considering a program complete.

Q90. Challenge | Shopping bill [Standard]

Read the price and quantity of an item. Calculate subtotal, 18% GST, and final bill using a named macro constant. Print a neatly formatted bill with two decimal places.

Suggested solution

```
#include <stdio.h>
#define GST_RATE 0.18
int main(void) {
    double price; int quantity;
    scanf("%lf%d", &price,&quantity);
    double subtotal=price*quantity, gst=subtotal*GST_RATE;
    printf("Subtotal: %.2f\nGST: %.2f\nFinal: %.2f\n", subtotal,gst,subtotal+gst);
    return 0;
}
```

Q91. Challenge | Student result slip [Challenge]

Read a student's name and marks in five subjects. Print the name, total, and average. Validate that each mark is between 0 and 100; if decision statements are not covered, write the validation as pseudocode.

Suggested solution

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char name[80]; double mark[5], total=0;
    if (!fgets(name,sizeof name,stdin)) return 1;
    name[strcspn(name,"\n")]='\0';
    for (int i=0;i<5;i++) {
        if (scanf("%lf", &mark[i])!=1 || mark[i]<0 || mark[i]>100) {
            puts("Invalid mark"); return 1;
        }
        total += mark[i];
    }
    printf("Student: %s\nTotal: %.2f\nAverage: %.2f\n", name,total,total/5.0);
    return 0;
}
```

Q92. Challenge | Unit converter [Basic]

Read a distance in kilometres and convert it to metres and centimetres. Use clear variable names, suitable types, and labeled output.

Suggested solution

```
#include <stdio.h>
int main(void) {
    double km; scanf("%lf", &km);
    printf("Metres = %.2f\nCentimetres = %.2f\n", km*1000.0,km*100000.0);
    return 0;
}
```

Q93. Challenge | Electricity estimate [Standard]

Read units consumed and a per-unit charge. Calculate the energy charge, add a fixed service fee defined with #define, and print a two-decimal bill.

Suggested solution

```
#include <stdio.h>
#define SERVICE_FEE 25.0
int main(void) {
    double units, rate; scanf("%lf%lf", &units,&rate);
    double energy=units*rate;
    printf("Energy: %.2f\nService fee: %.2f\nTotal: %.2f\n",
           energy,SERVICE_FEE,energy+SERVICE_FEE);
    return 0;
}
```

Q94. Challenge | Temperature converter [Basic]

Read a temperature in Celsius and convert it to Fahrenheit using F = (C * 9 / 5) + 32. Ensure the calculation does not lose its fractional part.

Suggested solution

```
#include <stdio.h>
int main(void) {
    double c; scanf("%lf", &c);
    printf("Fahrenheit = %.2f\n", c*9.0/5.0+32.0);
    return 0;
}
```

Q95. Challenge | Two-number report [Standard]

Read two integers and display their sum, difference, product, integer quotient, remainder, and the results of all six relational comparisons. Handle a zero divisor.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int a,b; scanf("%d%d", &a,&b);
    printf("sum=%d difference=%d product=%d\n",a+b,a-b,a*b);
    if (b!=0) printf("quotient=%d remainder=%d\n",a/b,a%b);
    else puts("quotient/remainder undefined for divisor zero");
    printf("< %d > %d <= %d >= %d == %d != %d\n",
           a<b,a>b,a<=b,a>=b,a==b,a!=b);
    return 0;
}
```

Q96. Challenge | Character report [Challenge]

Read one character and display the character, its integer character-code value, and whether it matches one of the lowercase vowels. Use getchar() or scanf() appropriately.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int ch=getchar();
    if (ch==EOF) return 1;
    printf("Character: %c\nCode: %d\n",ch,ch);
    if (ch=='a'||ch=='e'||ch=='i'||ch=='o'||ch=='u') puts("Lowercase vowel");
    else puts("Not a lowercase vowel");
    return 0;
}
```

Q97. Challenge | Modular geometry [Standard]

Write a C program with separate functions for rectangle area and perimeter. Read dimensions in main(), print both outputs, and explain which header is needed for standard input/output.

Suggested solution

```
#include <stdio.h>
double area(double l,double w) { return l*w; }
double perimeter(double l,double w) { return 2*(l+w); }
int main(void) {
    double l,w; scanf("%lf%lf", &l,&w);
    printf("Area=%.2f Perimeter=%.2f\n",area(l,w),perimeter(l,w));
    return 0;
}
```

Q98. Challenge | Pointer-based update [Challenge]

Read an integer, pass its address to a function, and have that function double the stored value through a pointer. Print the value before and after the call.

Suggested solution

```
#include <stdio.h>
void double_value(int *value) { *value *= 2; }
int main(void) {
    int n; scanf("%d", &n);
    printf("Before: %d\n", n);
    double_value(&n);
    printf("After: %d\n", n);
    return 0;
}
```

Q99. Challenge | Test and repair a program [Challenge]

Write a program that reads two integers and prints their average. Then deliberately identify one likely compilation mistake, one runtime risk, and one logic mistake that could occur in a beginner's version, and show how you would test for them.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int a,b; scanf("%d%d", &a,&b);
    printf("Average = %.2f\n", (a+b)/2.0);
    return 0;
}
```

A missing semicolon or wrong format specifier is a compilation/diagnostic issue; dividing by zero is a runtime risk if added elsewhere; using integer division is a logic error. Test with 3 and 6 (expected 4.50), plus negative values.

9. Two short mock CAT 1 practice sets

Attempt each set under timed conditions. These are fresh combinations of the same covered concepts, not predictions of the actual exam paper.

Q100. Mock A - Q1 | 4 marks [Standard]

Write a complete C program that reads the radius of a circle and prints its area and circumference using a #define constant for PI.

Suggested solution

```
#include <stdio.h>
#define PI 3.141592653589793
int main(void) {
    double r; scanf("%lf", &r);
    printf("Area = %.2f\nCircumference = %.2f\n", PI*r*r,2*PI*r);
    return 0;
}
```

Q101. Mock A - Q2 | 4 marks [Standard]

Trace and give the output: int x = 8, y = 3; printf("%d %d %d\n", x / y, x % y, x > y && y != 0);.

Suggested solution

x/y = 8/3 = 2; x%y = 2; x>y is true and y!=0 is true, so their logical AND is 1. Output: 2 2 1.

Q102. Mock A - Q3 | 6 marks [Standard]

Write an algorithm or C program to find the largest of three integers. Your answer must handle ties; show two test cases.

Suggested solution

Initialize largest=a; compare b and c with largest and update it. For example, (4,9,2) gives 9 and (-5,-3,-8) gives -3.

Q103. Mock A - Q4 | 6 marks [Standard]

Read a non-negative integer and calculate its factorial using repetition. State the result for zero and give one boundary test.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int n; unsigned long long f=1;
    scanf("%d", &n);
    if (n<0 || n>20) return 1;
    for (int i=1;i<=n;i++) f*= (unsigned long long)i;
    printf("%llu\n",f);
    return 0;
}
```

Q104. Mock A - Q5 | 5 marks [Standard]

Repair scanf("%d", number); and printf("Average = %d", (a + b) / 2); for a program that reads an int number and prints the decimal average of two integer marks.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int number,a,b;
    scanf("%d", &number); /* use &number */
    scanf("%d%d", &a,&b);
    printf("Average = %.2f\n", (a+b)/2.0);
    return 0;
}
```

Q105. Mock B - Q1 | 4 marks [Standard]

Read an integer and display whether it is even or odd. Write a flowchart or C program and test a negative even number.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int n; scanf("%d", &n);
    puts(n%2==0 ? "Even" : "Odd");
    return 0;
}
```

Q106. Mock B - Q2 | 4 marks [Standard]

For a = 12 and b = 5, predict a & b, a | b, a << 1, and a >> 1. Show enough binary working to justify each answer.

Suggested solution

12=1100 and 5=0101. AND = 0100 = 4; OR = 1101 = 13; 12<<1 = 11000 = 24; 12>>1 = 0110 = 6.

Q107. Mock B - Q3 | 6 marks [Standard]

Define a GST rate with #define. Read an item price, calculate GST and final price, and print both to two decimal places.

Suggested solution

```
#include <stdio.h>
#define GST_RATE 0.18
int main(void) {
    double price; scanf("%lf", &price);
    printf("GST Amount: %.2f\nFinal Price: %.2f\n",price*GST_RATE,price*(1+GST_RATE));
    return 0;
}
```

Q108. Mock B - Q4 | 6 marks [Basic]

Write a program that reads a character and displays it with a short message. Use getchar()/putchar() or scanf()/printf(), and include the required header.

Suggested solution

```
#include <stdio.h>
int main(void) {
    int ch=getchar();
    if (ch!=EOF) { printf("You entered: "); putchar(ch); }
    return 0;
}
```

Q109. Mock B - Q5 | 5 marks [Standard]

A program compiles but prints a wrong average because all inputs and calculations use int. Fix the calculation, name the error type, and propose one test that exposes the bug.

Suggested solution

Use a floating-point result: double average = (a + b) / 2.0; then print with %.2f. The bug is logical (integer truncation). A test with 3 and 6 exposes it because the expected result is 4.50.